

TP4 – Shape-from-shading

Équation eikonale

Soit $\mathbf{P}$ un point de la surface d'une scène 3D opaque, dont l'image $\mathbf{p}$ a pour coordonnées $[x, y]^\top$ dans le repère image. La *loi de Lambert* exprime le niveau de gris $I(x, y)$ de $\mathbf{p}$ sous une forme très simple, qui est en fait une relation de proportionnalité :

$$I(x, y) = \rho(\mathbf{P}) \mathbf{n}(\mathbf{P}) \cdot \mathbf{s}(\mathbf{P}) \quad (1)$$

où $\rho(\mathbf{P}) \in \mathbb{R}^+$ désigne l'albédo de $\mathbf{P}$, $\mathbf{n}(\mathbf{P})$ la normale sortante unitaire en $\mathbf{P}$, et $\mathbf{s}(\mathbf{P})$ le vecteur d'éclairage en $\mathbf{P}$. Un éclairage uniforme *frontal* d'intensité ψ_0 a pour coordonnées, dans le repère caméra :

$$\mathbf{s}(\mathbf{P}) = [0, 0, -\psi_0]^\top \quad (2)$$

En supposant la projection orthogonale, c'est-à-dire en négligeant la perspective, le point $\mathbf{P}$ a pour coordonnées $\mathbf{P} = [x, y, z(x, y)]^\top$ dans le repère caméra. La normale $\mathbf{n}(\mathbf{P})$ s'exprime donc sous la forme suivante (voir cours) :

$$\mathbf{n}(\mathbf{P}) = \frac{1}{\sqrt{|\nabla z(x, y)|^2 + 1}} \begin{bmatrix} \nabla z(x, y) \\ -1 \end{bmatrix} \quad (3)$$

En supposant de plus que $\rho(\mathbf{P}) = \rho_0$ (albédo uniforme), nous tirons des équations (1), (2) et (3) :

$$I(x, y) = \frac{\rho_0 \psi_0}{\sqrt{|\nabla z(x, y)|^2 + 1}} \quad (4)$$

Comme $\rho_0 \psi_0 = \max\{I(x, y)\} = I_{\max}$, nous tirons enfin de (4) :

$$|\nabla z(x, y)| = \sqrt{\left[\frac{I_{\max}}{I(x, y)}\right]^2 - 1} \quad (5)$$

Cette équation, appelée *équation eikonale*, constitue le modèle le plus simple de *shape-from-shading*.

Résolution de l'équation eikonale

L'équation eikonale nous informe sur la norme de ∇z en chaque point $\mathbf{p} = [x, y]^\top$, mais pas sur sa direction ni son orientation. En d'autres termes, parmi les deux angles qui caractérisent la normale, seule sa colatitude θ (angle avec l'axe Oz) est connue : en effet, nous ne disposons d'aucune information sur l'azimut ϕ de la normale. L'expression (3) montre que ϕ est également l'angle polaire de $\nabla z(x, y)$ dans le repère image Oxy .

Une façon de rendre bien posée la résolution de l'équation eikonale (5) consiste à imposer deux contraintes à l'inconnue $z(x, y)$. La première contrainte est la *condition au bord* $z(x, y) = 0$ en chaque point du bord de l'image (le bord constitue donc une *courbe de niveau*). Le gradient de $z(x, y)$ en ces points est orthogonal au bord de l'image, ce qui permet de calculer la normale $\mathbf{n}(\mathbf{P})$, à une ambiguïté près. En effet, une ambiguïté résiduelle porte sur le sens de $\nabla z(x, y)$. La deuxième contrainte consiste à orienter $\nabla z(x, y)$ vers le centre de l'image. Il est donc possible de calculer une nouvelle courbe de niveau $z(x, y) = c$, où $c > 0$ est « petit ». En répétant l'opération de proche en proche, on peut alors calculer la solution $z(x, y)$ sur l'ensemble de l'image.

Comme l'hypothèse $z(x, y) = 0$ ne correspond généralement pas à la réalité (la scène 3D n'a pas toujours un bord plan!), il ne faut pas s'attendre à ce que cette solution soit réaliste, mais il s'agit d'une solution exacte de l'équation eikonale, appelée *solution de viscosité maximale*, qui reproduit fidèlement l'image initiale sous un éclairage frontal, à ceci près que la solution est différentiable *presque partout*. Cela se traduit par le fait que le relief reconstruit comporte généralement des arêtes, qui sont facilement visibles lors du rééclairage.

Vous allez programmer deux méthodes d'approximation discrète de la solution de viscosité maximale. La première de ces méthodes est itérative (cf. exercice 1), tandis que la deuxième, appelée *fast marching*, calcule directement la solution de proche en proche, par le biais de l'évolution d'un « front d'onde » (cf. exercice 2). Cette dernière méthode a été proposée par Tsitsiklis en 1995, puis reprise par Sethian en 1996.

Exercice 1 : résolution itérative du *shape-from-shading*

Pour trouver une approximation discrète de la solution de viscosité maximale de l'équation eikonale (5), on peut utiliser les approximations suivantes des deux dérivées partielles de z par *différences finies centrées* (il est rappelé qu'en Matlab, $x \equiv j$ et $y \equiv i$) :

$$\left(\frac{\partial z}{\partial x}\right)_{i,j} \approx \frac{z_{i,j+1} - z_{i,j-1}}{2} \quad ; \quad \left(\frac{\partial z}{\partial y}\right)_{i,j} \approx \frac{z_{i+1,j} - z_{i-1,j}}{2} \quad (6)$$

Si nous désignons par $f_{i,j}$ le membre droit de (5) calculé au pixel (i, j) , nous pouvons trouver une approximation discrète de la solution de viscosité maximale de l'équation (5) à l'aide du schéma numérique suivant :

$$z_{i,j}^{(k+1)} = z_{i,j}^{(k)} + a \left\{ f_{i,j} - \sqrt{\left(\frac{z_{i,j+1}^{(k)} - z_{i,j-1}^{(k)}}{2}\right)^2 + \left(\frac{z_{i+1,j}^{(k)} - z_{i-1,j}^{(k)}}{2}\right)^2} \right\} \quad (7)$$

où k désigne le pas de l'itération et $a > 0$ est un paramètre. En effet, si ce schéma converge vers z^* , alors :

$$z_{i,j}^* = z_{i,j}^* + a \left\{ f_{i,j} - \sqrt{\left(\frac{z_{i,j+1}^* - z_{i,j-1}^*}{2}\right)^2 + \left(\frac{z_{i+1,j}^* - z_{i-1,j}^*}{2}\right)^2} \right\} \quad (8)$$

En simplifiant (8) et en utilisant (6), il est facile de vérifier que z^* est bien solution de l'équation eikonale (5). Malheureusement, le schéma (7) est numériquement instable. On lui préfère un schéma de type « Lax-Friedrichs », où $z_{i,j}^{(k)}$ doit être remplacé par la moyenne de la profondeur de ses quatre plus proches voisins, dont on montre qu'il est stable dès lors que $a \leq 0,5$ (en pratique, on fixe a à la valeur 0,5) :

$$z_{i,j}^{(k+1)} = \frac{z_{i-1,j}^{(k)} + z_{i+1,j}^{(k)} + z_{i,j-1}^{(k)} + z_{i,j+1}^{(k)}}{4} + a \left\{ f_{i,j} - \sqrt{\left(\frac{z_{i,j+1}^{(k)} - z_{i,j-1}^{(k)}}{2}\right)^2 + \left(\frac{z_{i+1,j}^{(k)} - z_{i-1,j}^{(k)}}{2}\right)^2} \right\} \quad (9)$$

Écrivez la fonction `lax_friedrichs`, appelée par le script `exercice_1`, permettant de résoudre l'équation eikonale (5) par le schéma itératif (9), **qui doit être appliqué seulement aux pixels du masque**. En guise d'initialisation de la profondeur, vous pourrez choisir $\{z_{i,j}^{(0)}\}$ uniformément égal à 0. Comme cela a déjà été dit, la reconstruction 3D obtenue n'est pas une approximation réaliste du relief, mais l'image simulée par rééclairage frontal reproduit fidèlement l'image d'origine, à part le long d'arêtes dont la présence est inhérente aux solutions de viscosité.

Remarque – Afin d'éviter les divisions par 0 lors de l'estimation de $|\nabla z|$ au pixel (i, j) , vous observerez que le niveau de gris $I_{i,j}$ est remplacé par $\max\{I_{i,j}, 0.1\}$ lors du calcul du second membre $f_{i,j}$ de (5).

Algorithme à marche rapide (*fast marching*)

Le code de l'exercice 1, qui est itératif, est trop lent pour pouvoir être appliqué à une image de grande taille, par exemple `Lena.png`. Il est nécessaire d'opter pour une méthode de résolution de l'équation eikonale plus rapide, en l'occurrence l'algorithme *fast marching*. Cet algorithme, qui simule l'évolution d'un « front », a été spécifiquement conçu pour résoudre l'équation eikonale (pas seulement celle du *shape-from-shading*). Il est rapide car non itératif, chaque pixel étant « visité » une seule fois.

Au démarrage de l'évolution, le front doit être connu : on y impose la condition au bord $z(x, y) = 0$. Les pixels sont répartis en trois classes, qui forment une partition de l'image :

- La classe « calculés » contient les pixels où la valeur de z a déjà été calculée.
- La classe « proches » contient les pixels non encore calculés, dont au moins un des 8 plus proches voisins a déjà été calculé.
- La troisième classe contient tous les autres pixels.

L'astuce de cet algorithme consiste à visiter les pixels dans un ordre tel que **les valeurs de z forment une suite croissante**. Le prochain pixel où l'inconnue z est calculée doit appartenir à la classe « proches ». Une fois le calcul effectué, ce pixel passera dans la classe « calculés ». Il faut commencer par calculer, pour chaque pixel $\mathbf{p}_1$ de la classe « proches » :

$$z_{\min}(\mathbf{p}_1) = \min_{\mathbf{p}_2 \in \mathcal{V}(\mathbf{p}_1)} \{z(\mathbf{p}_2) + \delta z(\mathbf{p}_1, \mathbf{p}_2)\} \quad (10)$$

où $\mathcal{V}(\mathbf{p}_1)$ contient les pixels voisins de $\mathbf{p}_1$ appartenant à la classe « calculés », et où $\delta z(\mathbf{p}_1, \mathbf{p}_2)$ est égal à la distance entre $\mathbf{p}_1$ et $\mathbf{p}_2$, qui vaut 1 ou $\sqrt{2}$ si l'unité de longueur est le pixel, multipliée par la moyenne des gradients de z en $\mathbf{p}_1$ et en $\mathbf{p}_2$:

$$\delta z(\mathbf{p}_1, \mathbf{p}_2) = \|\mathbf{p}_2 - \mathbf{p}_1\| \frac{|\nabla z(\mathbf{p}_1)| + |\nabla z(\mathbf{p}_2)|}{2} \quad (11)$$

Pour déterminer le prochain pixel $\widehat{\mathbf{p}}_1$ passant dans la classe « calculés », il faut faire une nouvelle recherche de minimum :

$$\widehat{\mathbf{p}}_1 = \underset{\mathbf{p}_1}{\operatorname{argmin}} \{z_{\min}(\mathbf{p}_1)\} \quad (12)$$

Intéressons-nous, dans un premier temps, à l'équation eikonale suivante, plus simple que (5) :

$$|\nabla z(x, y)| = 1 \quad (13)$$

La résolution de (13) permet de calculer la distance de chaque point sur la carte de France à la frontière la plus proche, et donc de déterminer l'endroit le plus éloigné de toute frontière. Lancez le script `carte_France`, qui calcule et affiche ce point sur la carte de France à l'aide de l'algorithme *fast marching*.

Exercice 2 : résolution directe du *shape-from-shading*

Écrivez la fonction `fast_marching`, appelée par le script `exercice_2`, permettant de calculer l'approximation discrète de la solution de viscosité maximale de l'équation eikonale (5). Les n_{paires} paires de pixels voisins sont stockées dans une matrice de taille $n_{\text{paires}} \times 2$, de nom `voisins`, dont chaque ligne contient les indices absolus d'une paire de pixels. Les distances entre pixels voisins sont stockées dans le vecteur `distances`; de taille $n_{\text{paires}} \times 1$.

Conseils de programmation

- Veillez à ne pas utiliser de boucle `for`, mais une boucle `while` (en l'occurrence : tant que la classe « proches » n'est pas vide).
- Il est conseillé de manipuler les pixels par leurs indices absolus, et d'utiliser deux vecteurs de booléens comportant autant d'éléments qu'il y a de pixels dans l'image, pour tenir à jour les listes de pixels appartenant aux classes « calculés » et « proches ».
- La boucle `while` de la fonction `fast_marching` doit avoir la structure suivante :
 1. Recherche du pixel $\mathbf{p}$ de la classe « proches » de plus faible profondeur.
 2. Le pixel $\mathbf{p}$ passe de la classe « proches » à la classe « calculés », et doit prendre la couleur de la classe « calculés ».
 3. Mise à jour (10) de la profondeur des voisins de $\mathbf{p}$ qui étaient déjà dans la classe « proches ». Pour cela, vous pouvez utiliser la séquence suivante, où `ind_p` est l'indice absolu de $\mathbf{p}$:


```
voisins_p = find(M_voisins(:, ind_p));
voisins_deja_dans_P = voisins_p(proches(voisins_p));
```
 4. Mise à jour de la classe « proches » et modification des couleurs.
 5. Mise à jour (10) de la profondeur des nouveaux pixels de la classe « proches ».
 6. Mise à jour de l'affichage tous les 100 tours de boucle.